Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch Introduction to Colab or Colab Features You May Have Missed to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see Overview of Colab. To create a new Colab notebook you can use the File menu above, or use the following link: create a new Colab notebook.

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see jupyter.org.

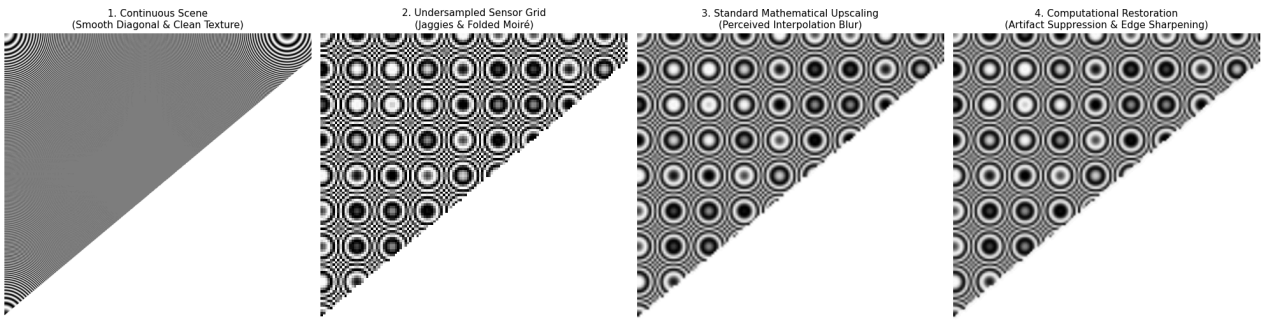
Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under Working with Data.

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

# 1. Generate a High-Resolution "Continuous" Scene
# We create a scene with both a sharp high-contrast diagonal (to show jaggies)
# and a tight chirped grid texture (to show Moiré frequency folding).
grid_size = 1024
y, x = np.ogrid[0:grid_size, 0:grid_size]

# Create a high-frequency repetitive zone-plate wave pattern
texture = np.sin((x**2 + y**2) / 300)
texture = ((texture - texture.min()) / (texture.max() - texture.min()) * 255)

# Create a sharp high-contrast diagonal geometric splitp
diagonal_mask = (x + 1.2 * y) > (grid_size * 1.1)
high_res_scene = texture.copy()
high_res_scene[diagonal_mask] = 255  # Pure white block meeting a complex texture

high_res_scene = high_res_scene.astype(np.uint8)

# 2. Simulate Improper Spatial Sampling (Undersampling)
# Downsampling sharply by picking every 8th pixel without an OLPF filter
sampling_factor = 8
undersampled_raw = high_res_scene[::sampling_factor, ::sampling_factor]
```

```
# 3. Simulate Standard Coarse Reconstruction (Bilinear Interpolation)
# This scales it back up, but results in severe "Perceived Blur" and jagged stair-stepping
standard_reconstruction = cv2.resize(undersampled_raw, (grid_size, grid_size), interpolation=cv2.INTER_LINEAR)

# 4. Computational Correction Pipeline
# Step A: Bilateral Filtering to clear false folded low-frequency Moiré ripples without blurring edge boundaries
denoised = cv2.bilateralFilter(standard_reconstruction, d=11, sigmaColor=85, sigmaSpace=85)

# Step B: High-frequency Edge Boosting (Simulated Structural Reconstruction Kernel)
kernel = np.array([[-1,-1,-1],
                   [-1,  9,-1],
                   [-1,-1,-1]])
computational_correction = cv2.filter2D(denoised, -1, kernel)

# 5. Plot and Compare the Results
plt.figure(figsize=(20, 10))

# Plot Original High-Res Scene
plt.subplot(1, 4, 1)
plt.imshow(high_res_scene, cmap='gray')
plt.title("1. Continuous Scene\n(Smooth Diagonal & Clean Texture)", fontsize=11)
plt.axis('off')

# Plot Undersampled Sensor Capture
plt.subplot(1, 4, 2)
plt.imshow(undersampled_raw, cmap='gray')
plt.title("2. Undersampled Sensor Grid\n(Jaggies & Folded Moiré)", fontsize=11)
plt.axis('off')

# Plot Standard Upscaling
plt.subplot(1, 4, 3)
plt.imshow(standard_reconstruction, cmap='gray')
plt.title("3. Standard Mathematical Upscaling\n(Perceived Interpolation Blur)", fontsize=11)
plt.axis('off')

# Plot Computational Correction
plt.subplot(1, 4, 4)
plt.imshow(computational_correction, cmap='gray')
plt.title("4. Computational Restoration\n(Artifact Suppression & Edge Sharpening)", fontsize=11)
plt.axis('off')

plt.tight_layout()
plt.show()
```



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including GPUs and TPUs, regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like RAPIDS cuDF that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the 10 minute guide or US stock market data analysis demo.

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just a few lines of code.

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks